

# oauth2-passkey

**Passwordless Authentication Library for Rust**

**Kimitoshi Takahashi | Tokyo Rust Show & Tell | 2026/03/31**

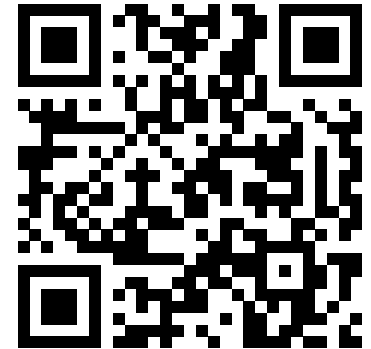
`crates.io/crates/oauth2-passkey`

`crates.io/crates/oauth2-passkey-axum`

# Live Demo

# Demo

1. **Google OAuth2** - First-time login with Google
2. **Passkey Promotion** - Library prompts to register fingerprint/face
3. **Passkey-only Login** - Next time, just biometrics. No redirect.
4. **Account Linking** - Both methods, same user



[passkey-demo.ccmp.jp](https://passkey-demo.ccmp.jp)

# What the Demo Showed

| Feature           | Details                                                                        |
|-------------------|--------------------------------------------------------------------------------|
| OAuth2/OIDC       | Google login (also works with FedCM)                                           |
| Passkey Promotion | After OAuth2 login, prompts user to register Passkey                           |
| Passkey           | Google Password Manager, Apple, Windows Hello, bitwarden, Proton Pass, YubiKey |
| Account Linking   | OAuth2 + Passkey mapped to same user                                           |
| Built-in UI       | Login page, account management, admin panel included                           |
| Session           | Cookie-based session with CSRF protection                                      |

All handled by a single library: `oauth2-passkey`

# Motivation

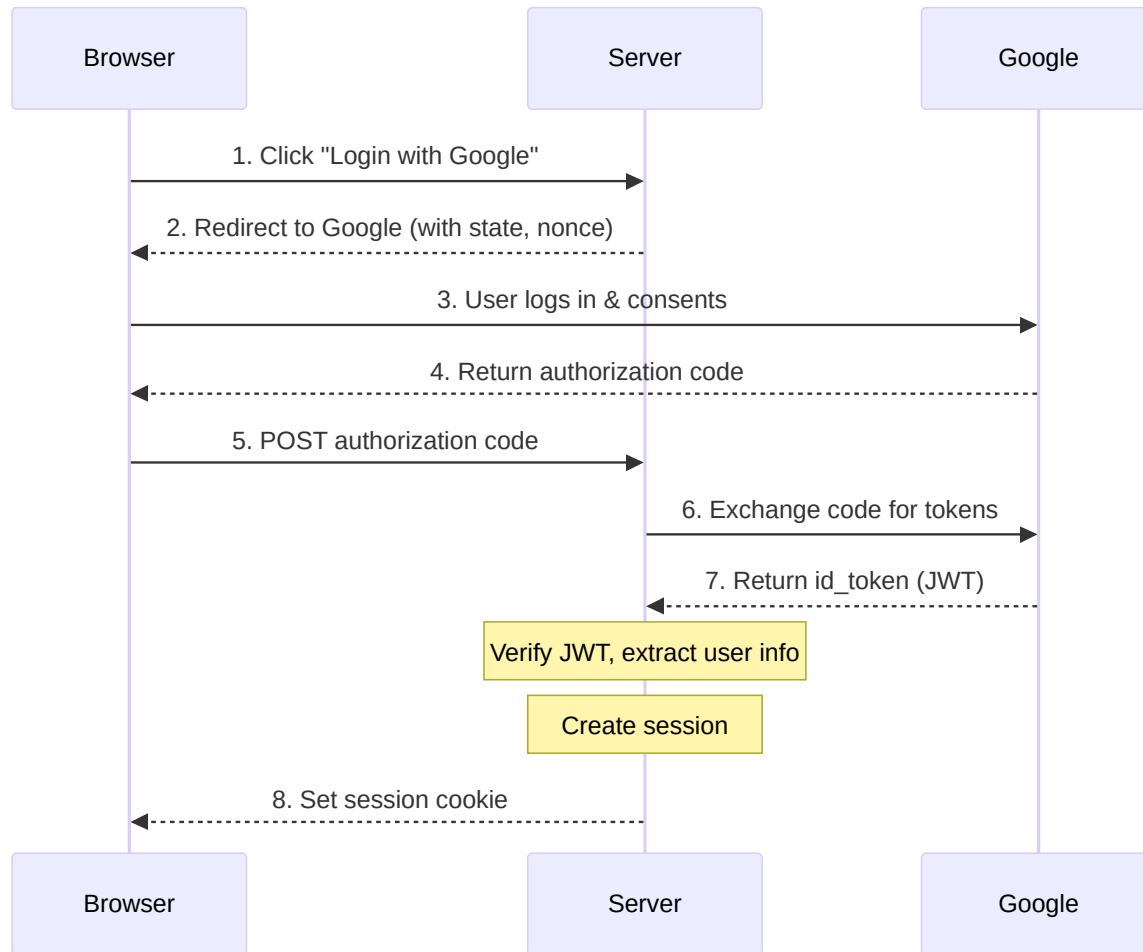
I wanted to build **exactly what you just saw** - and do it myself in Rust.

- Existing Rust ecosystem:
  - `webauthn-rs` - WebAuthn only
  - Various OAuth2 crates - OAuth2 only
  - **No combined solution** with session management
- So I built one. Published on **crates.io**.

# Agenda

1. **How OAuth2 & Passkey Work** - What happened behind the demo
2. **Using the Library** - init, extractor, middleware
3. **Storage & LazyLock** - Multi-DB support, why not Axum State
4. **Integrating with Your App** - Linking your data to auth users
5. **Wrap-up** - Summary & links

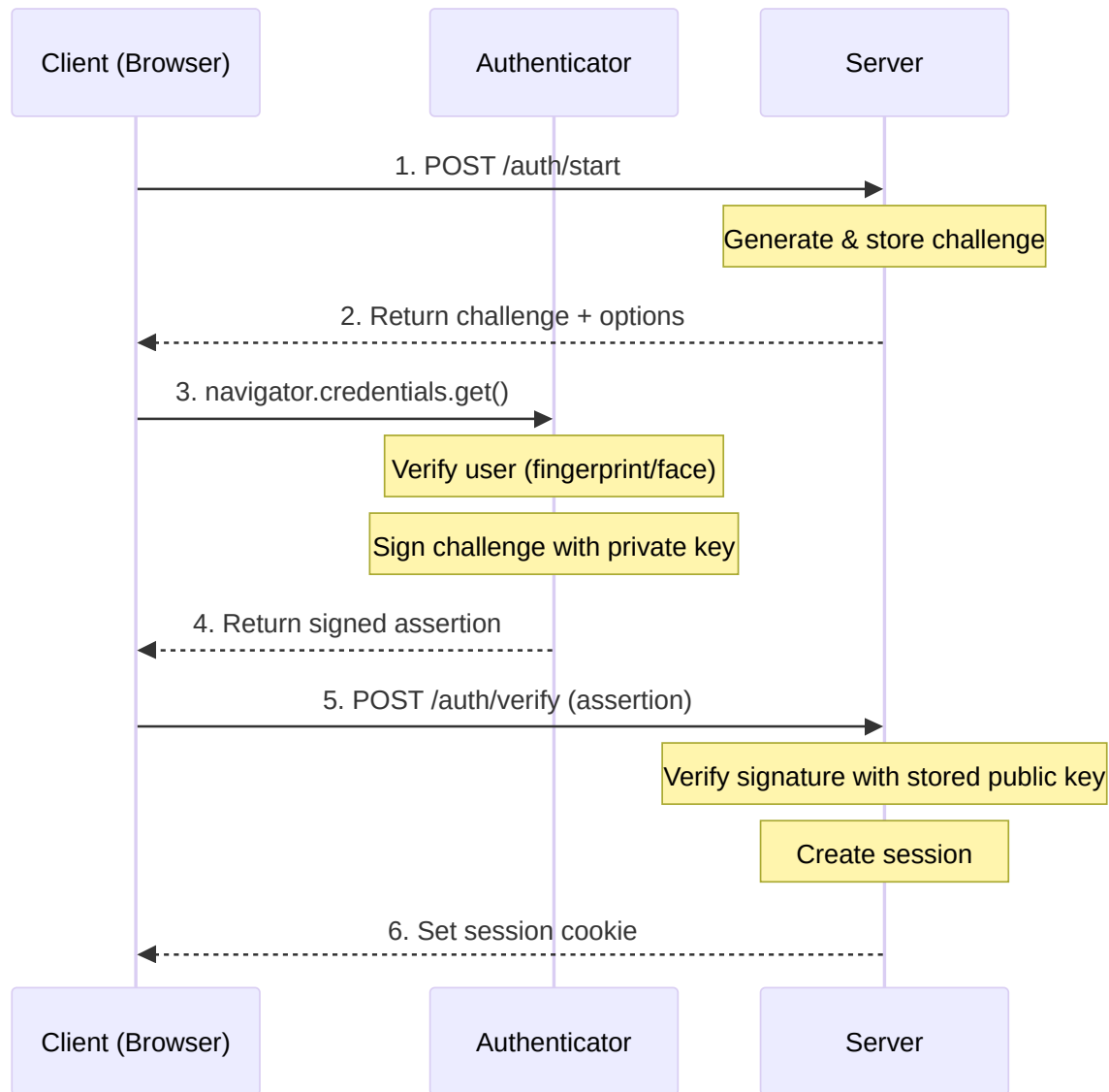
# How OAuth2/OIDC Works



## Page-redirect based auth:

1. User clicks "Login with Google"
2. Redirect to Google consent screen
3. Google returns authorization code
4. Server exchanges code for **id\_token** (JWT)
5. Extract user info, create session
6. Set session cookie

# How Passkey/WebAuthn Works



JavaScript-driven, no redirects:

1. Server generates **challenge**
2. Browser calls  
`navigator.credentials.get()`
3. Authenticator signs challenge with **private key**
4. Server verifies with stored **public key**
5. Create session, set cookie

Authenticators: Google Password Manager, YubiKey, Touch ID, Windows Hello



# Using the Library

# Environment Variables (.env)

```
ORIGIN='http://localhost:3001'

# Google OAuth2 credentials
OAUTH2_GOOGLE_CLIENT_ID='xxx.apps.googleusercontent.com'
OAUTH2_GOOGLE_CLIENT_SECRET='xxx'

# SQLite + in-memory cache (no DB setup required)
GENERIC_DATA_STORE_TYPE=sqlite
GENERIC_DATA_STORE_URL='sqlite:/tmp/auth.db'
GENERIC_CACHE_STORE_TYPE=memory
GENERIC_CACHE_STORE_URL='memory'
```

Swap to PostgreSQL/MySQL by changing `DATA_STORE_TYPE` and `URL` .

## Setup: init + merge

```
use oauth2_passkey_axum::{
    AuthUser, oauth2_passkey_full_router,
};

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    dotenv().ok();
    oauth2_passkey_axum::init().await?;          // 1. Initialize

    let app = Router::new()
        .route("/", get(index))
        .merge(oauth2_passkey_full_router()); // 2. Merge router

    // Auth endpoints are now at /o2p/*
    spawn_http_server(3001, app).await?;
    Ok(())
}
```

Built-in login UI, account management, and admin panel included.

# Page Protection: AuthUser Extractor

```
// Required auth - redirects to login if not authenticated
async fn protected(user: AuthUser) -> impl IntoResponse {
    format!("Hello, {}!", user.label)
}

// Optional auth - allows anonymous access
async fn public(user: Option<AuthUser>) -> impl IntoResponse {
    match user {
        Some(u) => format!("Hello, {}!", u.label),
        None    => "Hello, anonymous!".to_string(),
    }
}
```

Implemented via Axum's `FromRequestParts` trait:

- `AuthUser` -> auto-redirect on failure + DB query every time
- `Option<AuthUser>` -> no redirect, `None` for anonymous

Limitation: always redirects, always hits DB. What about APIs that need 401?

# Page Protection: Middleware

Middleware solves both: **choose 401 vs redirect**, and **skip DB when you don't need user info**.

```
let app = Router::new()
// Web page -> redirect to login
.route("/secret", get(handler)
  .route_layer(from_fn(is_authenticated_redirect)))

// API -> return 401 JSON
.nest("/api", api_router()
  .route_layer(from_fn(is_authenticated_401)));

// In handler: extract Attributes from Extension
fn h(Extension(user): Extension<AuthUser>)
fn h(Extension(csrf): Extension<CsrfToken>)
```

| Variant        | Unauth | DB? | Extension |
|----------------|--------|-----|-----------|
| _401           | 401    | No  | CsrfToken |
| _redirect      | Login  | No  | CsrfToken |
| _user_401      | 401    | Yes | AuthUser  |
| _user_redirect | Login  | Yes | AuthUser  |

# Storage & LazyLock Pattern

# Switch DB by Changing .env

```
# SQLite (dev/demo - no setup required)
GENERIC_DATA_STORE_TYPE=sqlite
GENERIC_DATA_STORE_URL='sqlite:/tmp/auth.db'

# PostgreSQL
GENERIC_DATA_STORE_TYPE=postgres
GENERIC_DATA_STORE_URL='postgres://user:pass@localhost:5432/mydb'

# MySQL / MariaDB
GENERIC_DATA_STORE_TYPE=mysql
GENERIC_DATA_STORE_URL='mysql://user:pass@localhost:3306/mydb'

# Cache: in-memory or Redis
GENERIC_CACHE_STORE_TYPE=memory          # or: redis
GENERIC_CACHE_STORE_URL='memory'         # or: redis://localhost:6379
```

No code changes. Just swap the env vars and restart.

# How It Works: env → LazyLock → DataStore trait

## 1. LazyLock reads env at startup:

```
static GENERIC_DATA_STORE:
    LazyLock<Mutex<Box<dyn DataStore>>>
    = LazyLock::new(|| {
        let db_type = env::var(
            "GENERIC_DATA_STORE_TYPE");
        let db_url = env::var(
            "GENERIC_DATA_STORE_URL");
        match db_type {
            "sqlite" => Box::new(
                SqliteDataStore { pool },
            "postgres" => Box::new(
                PostgresDataStore { pool },
            "mysql" => Box::new(
                MySqlDataStore { pool },
        }
    });
```

## 2. Callers dispatch via trait:

```
pub(crate) trait DataStore: Send + Sync {
    fn as_sqlite(&self)
        -> Option<&Pool<Sqlite>>;
    fn as_postgres(&self)
        -> Option<&Pool<Postgres>>;
    fn as_mysql(&self)
        -> Option<&Pool<MySql>>;
}

let store = GENERIC_DATA_STORE
    .lock().await;
match (store.as_sqlite(),
        store.as_postgres(),
        store.as_mysql()) {
    (Some(pool), _, _) =>
        do_sqlite(pool).await?,
    (_, Some(pool), _) =>
        do_postgres(pool).await?,
    ...
}
```



# Why LazyLock Instead of Axum State?

## If the library used State:

```
// User must compose AuthState into AppState
struct AppState {
    auth: AuthState, // library's state
    pool: PgPool,    // user's own state
}
let app = Router::new().with_state(state);
```

User: manage state composition

Library: thread state through 80+ internal functions

LazyLock: simpler for both library users and library internals.

## oauth2-passkey: LazyLock globals

```
// User just calls init()
oauth2_passkey_axum::init().await?;

// Library internally:
let store = GENERIC_DATA_STORE
    .lock().await;
// No state parameter needed
```

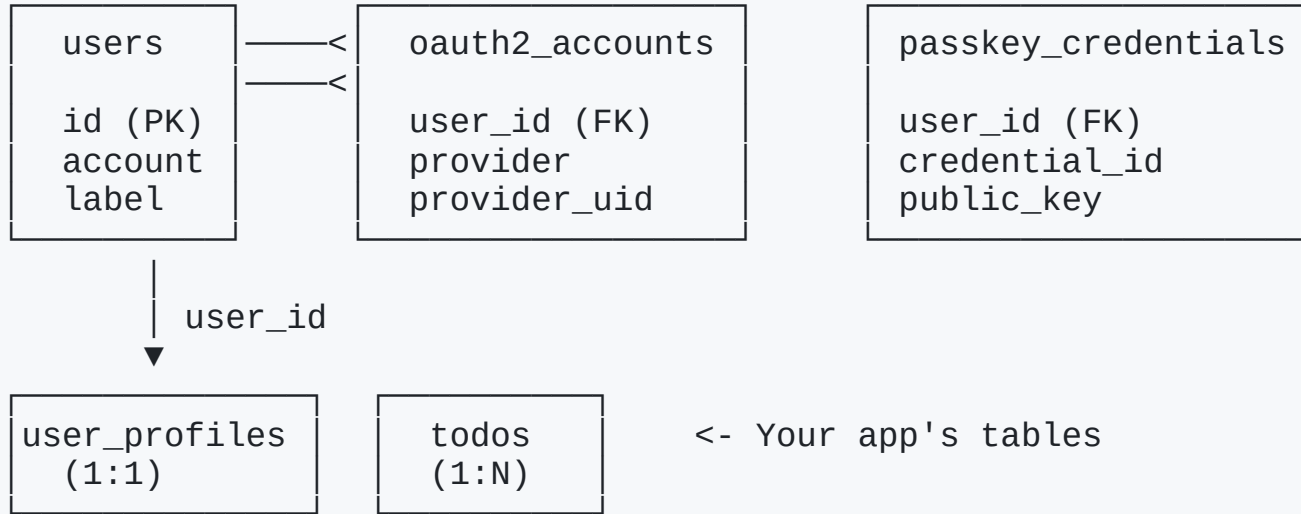
User: just call `init()`, done

Library: any function accesses storage directly

# Integrating with Your App

# Account Linking: Internal DB Structure

oauth2-passkey manages these tables:



One user can have multiple OAuth2 accounts AND multiple passkeys.

# Your App's Data: Link via AuthUser.id

## Setup: two databases, one app

```
// 1. oauth2-passkey init
oauth2_passkey_axum::init().await?;

// 2. App's own DB
let pool = db::init_db().await?;
let state = AppState { pool };

// 3. Combine routes
let app = Router::new()
    .route("/", get(index))
    .merge(handlers::router())
    .with_state(state)
    .merge(oauth2_passkey_full_router());
```

## Handler: use user.id as FK

```
async fn create_todo(
    State(state): State<AppState>,
    user: AuthUser,
    Form(form): Form<TodoForm>,
) -> Result<Response, ...> {
    db::create_todo(
        &state.pool,
        &user.id, // link to auth user
        &form.title,
    ).await?;
    Ok(Redirect::to("/").into_response())
}
```

See `demo-profile` (1:1) and `demo-todo` (1:N).

# Wrap-up

# Summary

| What         | Details                                                        |
|--------------|----------------------------------------------------------------|
| Library      | <code>oauth2-passkey</code> + <code>oauth2-passkey-axum</code> |
| What it does | Passkey + OAuth2 auth for Axum apps                            |
| DB support   | SQLite, PostgreSQL, MySQL (via sqlx)                           |
| Key design   | LazyLock globals, DataStore trait dispatch                     |
| Integration  | <code>AuthUser.id</code> links your app data to auth           |

## Links

- **crates.io:** `oauth2-passkey` , `oauth2-passkey-axum`
- **GitHub:** [github.com/ktaka-ccmp/oauth2-passkey](https://github.com/ktaka-ccmp/oauth2-passkey)
- **X:** @ktaka



GitHub

# Thank You!

Questions?

Kimitoshi Takahashi (@ktaka) on X / GitHub

# About Me

- @ktaka (X / GitHub)
- Self-employed, reskilling in Rust
- Building web authentication libraries
- Third year writing Rust



[ktaka.blog.ccmp.jp/p/p.html](https://ktaka.blog.ccmp.jp/p/p.html)



## Extra Slides

# What is LazyLock?

`std::sync::LazyLock` — a value initialized **once**, on first access, then shared immutably. Thread-safe. Stable since Rust 1.80.

```
use std::sync::LazyLock;

// Evaluated once, on first access. Never again.
static CONFIG: LazyLock<String> = LazyLock::new(|| {
    std::env::var("MY_CONFIG").expect("MY_CONFIG must be set")
});

fn anywhere() {
    println!("{}", *CONFIG); // Just use it. No parameter needed.
}
```

Like `lazy_static!` but in std. No macro, no extra crate.

# LazyLock: Benefits & Trade-offs

## Benefits

- **Zero boilerplate for users** - no `AppState` struct to create
- **No state threading** - 80+ internal functions access storage directly
- **Env-only config** - switch DB by changing `.env`, no code changes
- **Fail-fast init** - `init().await?` panics on invalid env at startup

## Trade-offs

- Single instance per process (fine for auth library)
- Implicit dependencies (function signatures don't show DB access)
- Test isolation needs `#[serial]` (shared global state)

An intentional design trade-off: Rust purists would use `State`, but LazyLock keeps complexity inside the library.

# Middleware: Why Not Just Use the Extractor?

## AuthUser extractor:

- Always redirects on auth failure
- Always fetches user from DB
- Great for web pages that need user info

**Problem:** APIs should return 401 JSON, not redirect HTML. And sometimes you just need to check "is this user logged in?" without a DB query.

## Middleware gives 2 axes of control:

### *Response type:*

- `_redirect` → Web pages (GET redirects to login)
- `_401` → APIs (always returns 401)

### *Extension injected:*

- `_user` → DB query, `Extension<AuthUser>`
- Non-`_user` → No DB query, `Extension<CsrfToken>` only

```
// Handler with _user middleware
async fn page(
    Extension(user): Extension<AuthUser>,
) -> impl IntoResponse { ... }

// Handler with non-_user middleware
async fn api(
    Extension(csrf): Extension<CsrfToken>,
) -> impl IntoResponse { ... }
```

# Newtype Wrappers: Type Safety

```
pub struct Csrftoken(String);  
pub struct UserId(String);  
pub struct SessionId(String);  
pub struct SessionCookie(String);  
pub struct CsrftokenVerified(pub bool);  
pub struct AuthenticationStatus(pub bool);
```

Prevent mixing up strings/bools at compile time.

```
// Without newtypes - compiles but wrong!  
fn check(session_id: &str, user_id: &str);  
check(user_id, session_id); // Oops!  
  
// With newtypes - compile error!  
fn check(session_id: SessionId,  
        user_id: UserId);  
check(user_id, session_id); // Error!
```

# Newtype Wrappers: Validation in Constructors

```
impl UserId {  
    pub fn new(id: String) -> Result<Self, SessionError> {  
        if id.is_empty() {  
            return Err(SessionError::Validation("User ID cannot be empty".into()));  
        }  
        if id.len() > 255 {  
            return Err(SessionError::Validation("User ID too long".into()));  
        }  
        if !id.chars().all(|c| c.is_ascii_alphanumeric()  
            || matches!(c, '-' | '_' | '.' | '@' | '+')) {  
            return Err(SessionError::Validation("Invalid characters".into()));  
        }  
        Ok(UserId(id))  
    }  
}
```

If you have a `UserId`, it's guaranteed valid. No need to re-validate.

# Error Hierarchy with thiserror

```
#[derive(Error, Debug)]
pub enum CoordinationError {
    #[error("Unauthorized access")]
    Unauthorized,
    #[error("Resource not found: {resource_type} {resource_id}")]
    ResourceNotFound { resource_type: String, resource_id: String },

    // Wrap module-specific errors
    #[error("OAuth2 error: {0}")] OAuth2Error(OAuth2Error),
    #[error("Passkey error: {0}")] PasskeyError(PasskeyError),
    #[error("Session error: {0}")] SessionError(SessionError),
    #[error("User error: {0}")]   UserError(UserError),
}
```

Each module has its own error type. The coordination layer wraps them all.

# From Impls with Auto-Logging

```
impl From<OAuth2Error>
  for CoordinationError
{
  fn from(err: OAuth2Error) -> Self {
    let error = Self::OAuth2Error(err);
    tracing::error!("{}", error);
    error // Auto-log on conversion!
  }
}
```

Every `?` that crosses a module boundary automatically logs:

```
// ? triggers From impl + logging
let token = exchange_code(code)
    .await?; // OAuth2Error -> logged
let user = get_user(id)
    .await?; // UserError -> logged
```

No manual `tracing::error!()` needed.



# Security: Constant-Time Comparison

```
use subtle::ConstantTimeEq;

// CSRF token verification
if header_csrf_token
    .as_bytes()
    .ct_eq(session_csrf_token
        .as_str().as_bytes())
    .into()
{
    // Token matches
}
```

## Why not just `==` ?

- Regular `==` short-circuits on first mismatch
- Attacker measures response time to guess bytes
- `ct_eq` always takes the same time

Used for CSRF tokens, session validation, and other security-sensitive comparisons.

# Atomic SQL: No Explicit Transactions

```
// "Delete user, but only if they're not the last admin"
// Done in a SINGLE SQL statement - no transaction needed

pub async fn delete_user_if_not_last_admin_postgres(
    pool: &Pool<Postgres>, id: UserId,
) -> Result<bool, UserError> {
    let result = sqlx::query(&format!(
        r#"DELETE FROM {table}
           WHERE id = $1
           AND (SELECT COUNT(*) FROM {table}
                WHERE is_admin = true) > 1"#
    ))
    .bind(id.as_str())
    .execute(pool).await?;

    Ok(result.rows_affected() > 0)
}
```

Business logic in **SQL** = atomic by default. No race conditions.

# SQL Dialect Differences

## PostgreSQL - one query

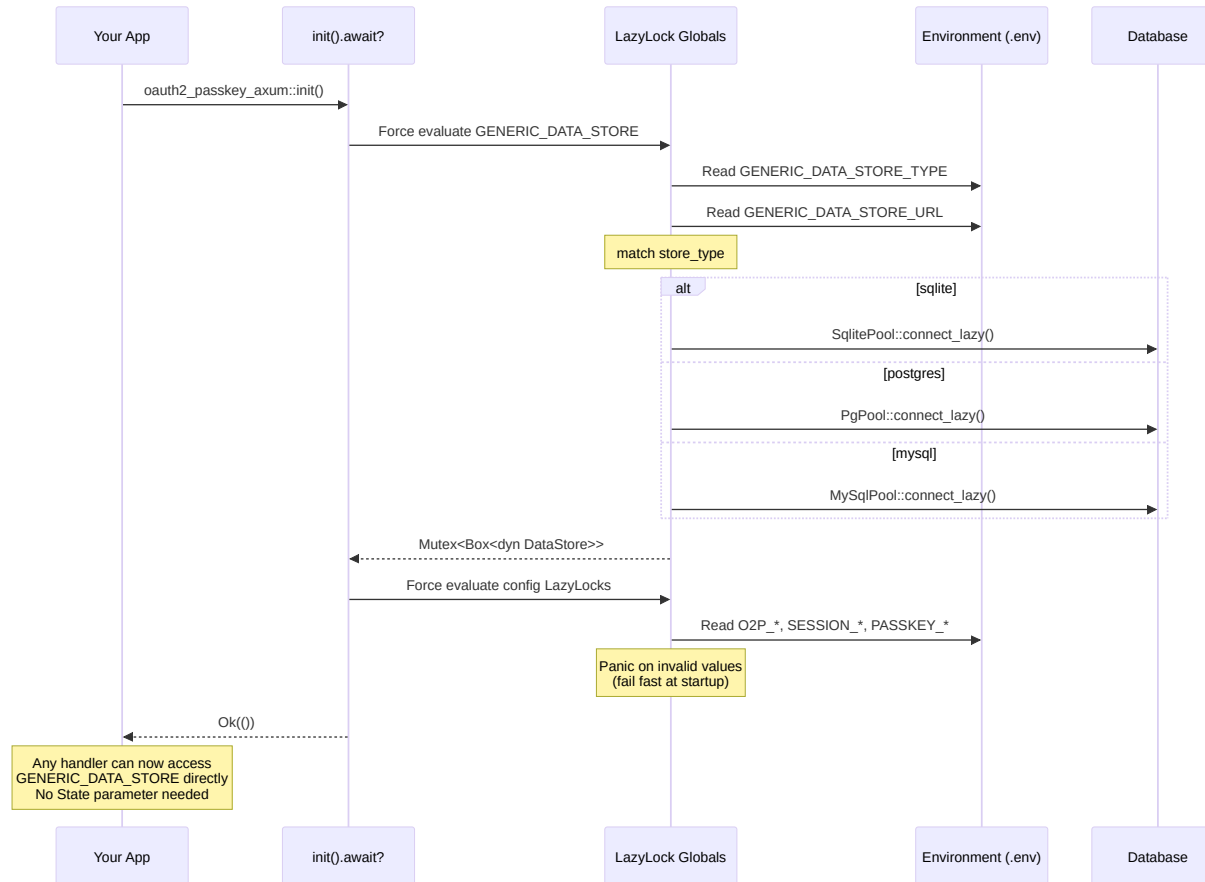
```
sqlx::query_as::<_, User>(&format!(  
    "INSERT INTO {table} (...)  
    VALUES ($1, $2, ...)  
    ON CONFLICT (id) DO UPDATE SET ...  
    RETURNING *"  
)).fetch_one(pool).await
```

## SQLite - need two queries

```
sqlx::query(&format!(  
    "INSERT INTO {table} (...)  
    VALUES (?, ?, ...)  
    ON CONFLICT (id) DO UPDATE SET ..."  
)).execute(pool).await?;  
  
sqlx::query_as::<_, User>(&format!(  
    "SELECT * FROM {table} WHERE id = ?"  
)).fetch_one(pool).await
```

Supporting 3 databases means handling these quirks per-backend.

# LazyLock Initialization Flow

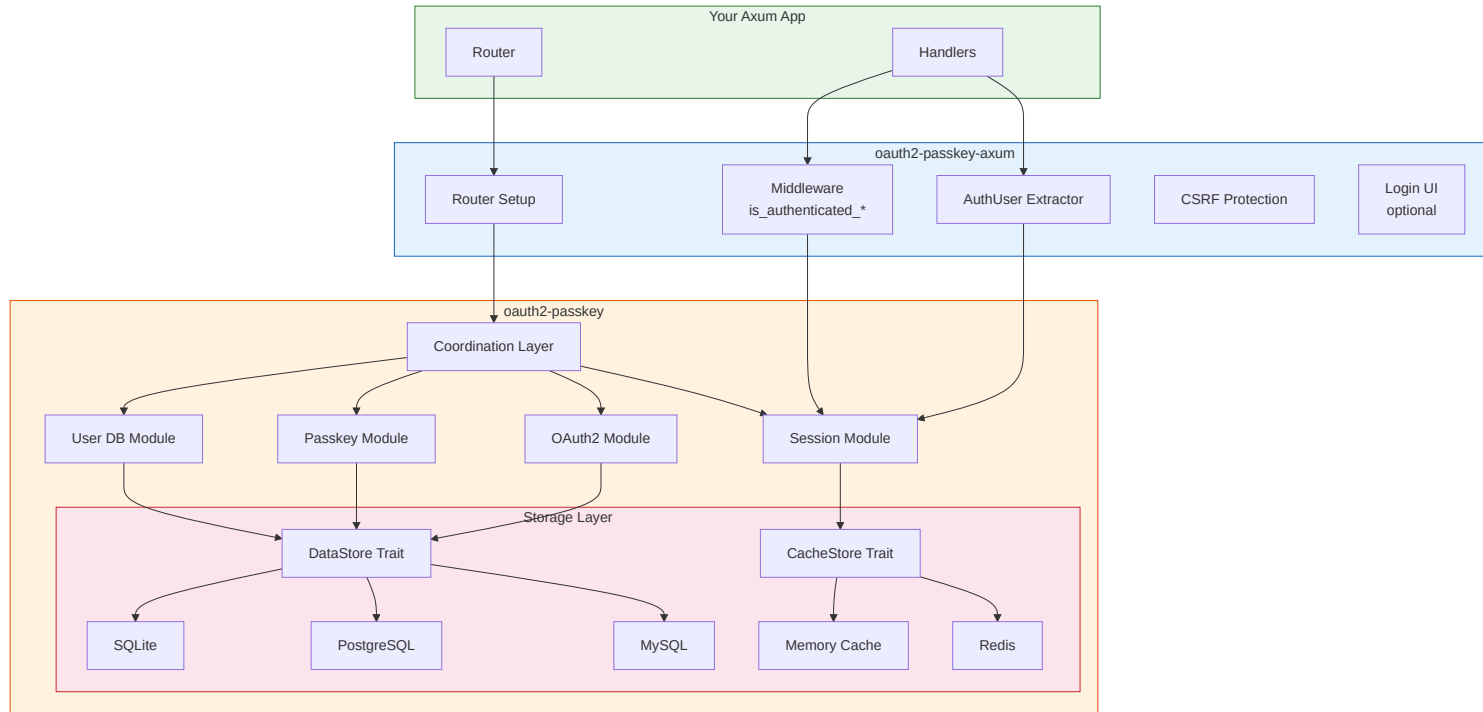


## Startup sequence:

1. App calls `init().await?`
2. Forces all `LazyLock` globals to evaluate
3. Reads env vars (`DB_TYPE`, `DB_URL`, ...)
4. Creates appropriate connection pool
5. Panics on invalid config (fail-fast)

After init, any internal function can access `GENERIC_DATA_STORE` directly - no state parameter needed.

# Crate Structure (Detail)



# From/Into: Clean Layer Boundaries

```
// DB layer -> Session layer
impl From<DbUser> for SessionUser {
    fn from(db_user: DbUser) -> Self {
        Self {
            id: db_user.id,
            account: db_user.account,
            label: db_user.label,
            is_admin: db_user.is_admin,
            ..
        }
    }
}
```

```
// Session -> Axum layer
impl From<SessionUser> for AuthUser {
    fn from(su: SessionUser) -> Self {
        AuthUser {
            id: su.id,
            // ... copy fields
            csrf_token: String::new(),
            session_id: String::new(),
            // ^ Axum-specific fields
        }
    }
}
```

Each layer has its own type. `From` impls make conversion seamless with `into()`.

# Future Plans

- **DPoP (Demonstration of Proof-of-Possession)**
  - Bind tokens to a cryptographic key
  - Prevent token theft/replay
- **Bearer Token support**
  - API authentication beyond session cookies
- **FedCM (Federated Credential Management)**
  - Browser-native identity UI (experimental, already partially implemented)
- **More OAuth2 providers**
  - GitHub, Apple, Microsoft, etc.